

总结

选用 janim (<https://janim.readthedocs.io/zh-cn/latest/>) 作为动画生成引擎

首先使用

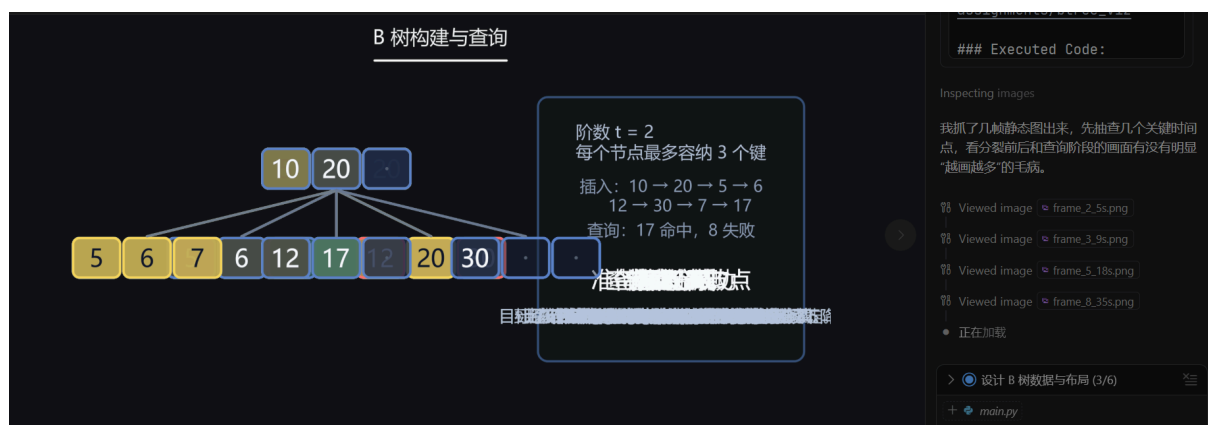
/create-skill <https://janim.readthedocs.io/zh-cn/latest>

让 Copilot 读取分析 janim 的文档，随后调用 skill

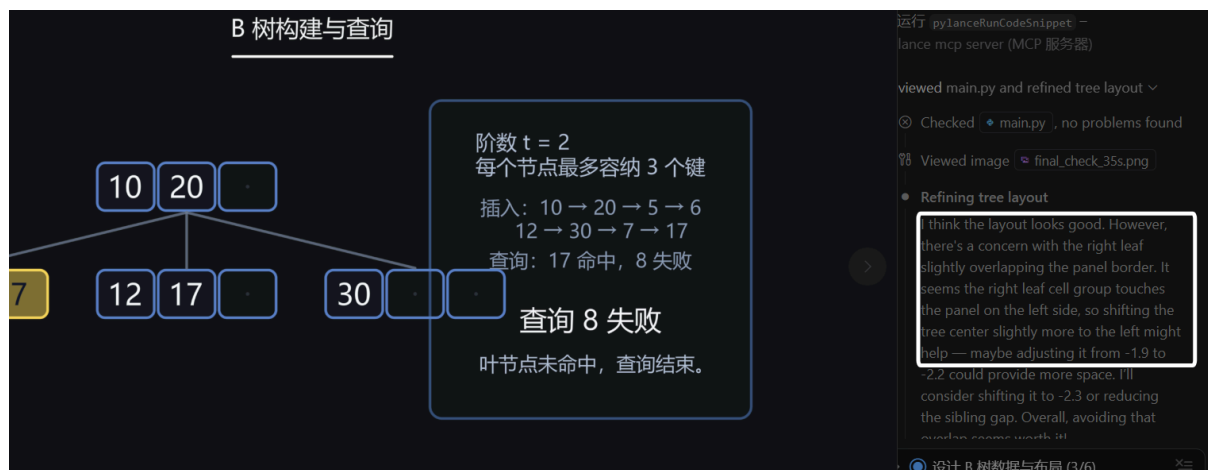
/janim-workflow 编写动画呈现b树的构建操作、动态变化过程、查询过程

生成可视化代码文件

向 ai 反馈并让其尝试修复了部分布局偏移、对象重叠的问题并成功修复。



Copilot 根据 skill 自行编写代码导出了关键帧进行查看，并感知到了问题所在



最后手动调整了部分动画的时长、缓动这类无法被 AI 感知的细节问题，结束。

完整对话过程如下：

Create Skill Step

User: /create-skill <https://janim.readthedocs.io/zh-cn/latest/>

GitHub Copilot: Optimizing tool selection...我先把相关说明和 JAnim 文档脉络摸清，再起草一个可复用的 SKILL.md，目标是把它落成到当前工作区里并尽量做到可直接用。

已提取 <https://janim.readthedocs.io/zh-cn/latest/>

Read skill agent-customization

我已经拿到定制化规范和 JAnim 文档首页；下一步把技能模板、当前项目上下文和 JAnim 关键章节拼起来，做成一个靠谱的初稿。

Read

Read

已创建 5 个待办事项

Read skill agent-customization:

Read

Read

现在项目侧信息也齐了：这是个极简 JAnim 项目。我再抓几页关键文档，把技能说明写得“既会找路，又会动手”，避免只会背目录不会干活。

已提取 7 资源

我开始落地技能文件了：做成工作区范围、偏 JAnim 实战工作流的版本，先保证它现在就能用，后面再根据你的偏好微调。

Created

技能草稿已经写进工作区了；我现在做一次快速自检，确认 frontmatter、路径和内容都没有“看起来像 YAML、实际上像薛定谔配置”的问题。

Checked , no problems found

Read

Read skill janim-workflow

草稿通过自检了，我接着把最需要你拍板的地方浓缩成三个小问题，这样后续要么我帮你精修，要么你直接开用都行。

正在分析你的回答...

你已经替这个技能定了方向：工作区内使用、兼顾通用 JAnim 和 B 树可视化、还要补参考资料。我这就把它升级成“主流程 + 两份随查手册”的版本。

Created

Created

主流程和两份参考手册都准备好了；我再把入口文件串起来，让技能正文能显式指向这些附加资料。

Generating patch (63 lines) in

我做最后一轮检查，确保主技能和两份参考资料都能在工作区里干净落地，然后就给你总结怎么用。

Checked , , no problems found

Read

Read skill janim-workflow

收尾一下进度板：主体技能、补充参考、校验和可用示例都已经就位，我把清单标成完成后给你一个精炼总结。

已完成：读取技能模板规范 (1/5)

已按 #prompt:SKILL.md 的流程把它做成了一个工作区级 **JAnim** 技能，并且根据你刚才的选择，把它强化成了通用 **JAnim** 开发 + B 树/算法可视化的版本。顺手还补了两份参考资料，避免技能只会“讲流程”不会“指路”。

已落地的文件

- SKILL.md
主技能文件，定义触发词、适用场景、分支逻辑、完成标准和示例提示词。
- docs-map.md
JAnim 官方中文文档地图：把“问题类型”快速映射到对应文档页面。
- btree-viz-playbook.md
面向当前 btree_viz 仓库的 B 树可视化手册：讲布局、过渡、节点分裂、查找路径、教学节奏。

这个技能现在会产出什么

它现在能比较稳定地处理这几类请求：

- JAnim 动画编写与修改
- Timeline / construct() / play() / forward() / .anim 等 API 解释
- TypstText / TypstMath / TypstDoc 的排版与索引
- GUI 预览、热重载、子物件选择相关工作流
- uv run janim ... 风格的当前仓库运行方式
- B 树、平衡树、搜索路径、节点分裂等算法可视化场景
- Manim 到 JAnim 的思路迁移

而且它会优先参考我已经核对过的官方页面，包括：

- installation.html
- get_started.html
- tutorials/animations.html
- tutorials/typst_usage.html
- tutorials/use_gui.html
- janim/imports.html
- other/faq.html

怎么试用

在聊天里可以直接调用 /janim-workflow，后面接你的需求。比如：

- /janim-workflow 用 JAnim 给 B 树插入 37 的过程做一个带节点分裂的教学动画，从 main.py 开始
- /janim-workflow 解释 main.py 里 Create、Transform、Uncreate 的区别，并顺便改进节奏
- /janim-workflow 把当前 TypstText 改成更适合公式展示的写法，并说明为什么用 TypstMath
- /janim-workflow 按 B 树可视化手册重构节点布局，让变换尽量用 Transform 而不是整棵树重画

我做过的校验

这次新增的 3 个文件都已检查过：

- 路径正确

- frontmatter 合法
- 内容可读
- 无错误报告

小结一句：现在它不是一个“空壳技能”，而是一个会先查官方文档、再结合当前 **repo** 的 **uv + JAnim + B 树可视化** 习惯来做事的版本。

接下来值得补的相关定制

如果你想把这个 workflow 继续打磨成“更像你自己的助手”，下一步最值的通常是这几个：

- `janim.instructions.md`
做成常驻指令，让代理在这个仓库里默认遵循：
 - 优先 `uv run janim ...`
 - 优先小步修改
 - 动画逻辑和树结构逻辑分离
 - B 树过渡尽量保留对象连续性
- `btree-scene.prompt.md`
做成一个一键场景脚手架提示，例如“生成插入动画骨架 / 生成分裂动画骨架”。
- `janim-review.agent.md`
做成一个专门审查 **JAnim** 动画质量的代理，重点看：
 - 动画节奏
 - Typst 选型
 - 布局函数是否统一
 - 是否过度重绘

当前清单

- [x] 读取技能模板规范
- [x] 梳理项目与 JAnim 上下文
- [x] 起草工作区 JAnim 技能
- [x] 校验技能文件位置与内容
- [x] 指出待确认项并给示例用法

如果你愿意，我下一步可以直接继续帮你补一个 `.github/instructions/janim.instructions.md`，把这个仓库的 JAnim/B 树约定变成默认行为。

已进行更改。

Implementation Step

User: `/janim-workflow` 编写动画呈现 b 树的构建操作、动态变化过程、查询过程

GitHub Copilot: Read memory

我先把项目结构、现有动画代码和仓库内的 JAnim 参考资料一起摸清，这样能尽快定位可复用部分并做出不“硬拼”的 B 树动画。

Read

Search complete: 勘查代码结构

Read

Read

Read skill janim-workflow:

Read skill janim-workflow:

Searched codebase for “btree_viz main.py README pyproject.toml github skills janim-workflow references”, no results

Searched for files matching `**/main.py`, 1 match

Searched for files matching `**/README.md`, 1 match

Searched for files matching `**/pyproject.toml`, 1 match

Searched for files matching `**/.github/skills/janim-workflow/references/**`, 2 matches

Searched for regex `B\树|btree|B 树可视化`, 6 results

Searched for regex `JAnim|janim|Timeline|construct|布局|动画节奏`, 109 results

Read

Read skill janim-workflow:

Read skill janim-workflow:

Read skill janim-workflow

我已经确认这是个极简的 `uv + janim[gui]` 项目，当前只有一个示例 `Timeline`；接下来先补上任务清单，再抓取官方文档里的动画机制与对象构建细节，避免凭印象写 API。

已创建 6 个待办事项

我现在去抓官方文档里最相关的页面，目标是确认 `Timeline/construct/play`、并行动画和常用图元的标准写法，顺手把可能要用到的后续链接也挖出来。

已提取 7 资源

我已经拿到核心节奏规则了；现在继续补抓“物件组/布局”和几个直接会用到的 API 页面，避免后面为了一个 `Group` 或 `Text` 方法跟树节点打架。

已提取 6 资源

我这边已经把 JAnim 的“打法手册”补齐了：`play/show/forward/prepare`、`AnimGroup/Succession`、`Group.arrange`、`Rect/Line/Text` 都有依据。下一步开始把 B 树的数据、布局和动画分层设计出来。

正在启动: 设计 B 树数据与布局 (3/6)

我准备开始动代码前，先把当前工作区的 Python/JAnim 运行环境对齐好，后面检查 API、跑预览和排错都会更稳。

Configuring a Python Environment Completed with input: { “resourcePath”: “e:\[data_deleted]\btree_viz” }

Completed with input: { “resourcePath”: “e:\[data_deleted]\btree_viz” }

我想用本地已安装的 JAnim 做一点轻量自省，确认若干类和方法是否真的存在，省得代码写完再和 API “面面相觑”。

已运行 `pylanceWorkspaceRoots` Completed with input: { “fileUri”: “file:///e:/[data_deleted]/btree_viz/main.py” }

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from janim.imports import *\n\nchecks = {\n 'Group': Group,\n 'Rect': Rect,\n 'RoundedRect': RoundedRect,\n 'Line': Line,\n 'Text': Text,\n 'Title': Title,\n 'Transform': Transform,\n 'Create': Create,\n 'FadeIn': FadeIn,\n 'FadeOut': FadeOut,\n 'Indicate': Indicate,\n 'AnimGroup': AnimGroup,\n 'Succession': Succession,\n 'Wait': Wait,\n 'Do': Do,\n 'ShowCreationThenDestructionAround': ShowCreationThenDestructionAround,\n}\n\nprint(sorted(checks))\n\nprint('Rect has points:', hasattr(Rect(), 'points'))\n\nprint('Text has points:', hasattr(Text('hi'), 'points'))\n\nprint('Group has points:', hasattr(Group(), 'points'))\n\nrect = Rect(2,1)\n\nprint('box center exists:', hasattr(rect.points, 'box'))\n\nprint('center', hasattr(rect.points.box, 'center'))\n\nfor name in ['shift', 'arrange', 'arrange_in_grid', 'to_border', 'to_center', 'next_to', 'move_to', 'scale']:\n print(name, hasattr(rect.points, name))\n }

我已经把结构想顺了：用真实的 B 树插入逻辑生成快照，用统一布局函数布点，再让节点/边和信息面板按步骤过渡。现在直接把 main.py 换成这个版本。

Generating patch (852 lines) in

我先做两层验证：一层看静态错误，一层直接在本地构建时间轴，尽早把 API 或类型上的小脾气揪出来。

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from main import BTreeBuildAndSearch\n\ntimeline = BTreeBuildAndSearch()\nbuilt = timeline.build(quiet=True)\n\nprint(type(built).name)\n\nprint(f'duration={timeline.current_time:2f}')\n }

Checked , no problems found

问题定位到了：不是逻辑，是字体配置。JAnim 这里把不存在的字体当成硬错误了，我先确认一个本机肯定可用的中文字体，再把配置收紧。

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from janim.imports import *\n\nfor font_name in ['Microsoft YaHei', 'Microsoft YaHei UI', 'SimHei', 'SimSun', 'Consolas']:\n try:\n Text('B 树', font=font_name)\n print('ok:', font_name)\n except Exception as exc:\n print('fail:', font_name, type(exc).name)\n }

Generating patch (8 lines) in

我去读一下对应片段，直接对准真实上下文改，不和补丁格式打架。

Searched for text CONFIG = Config(btree_viz · main.py), 1 result

Generating patch (9 lines) in

字体问题已经处理掉一半，我继续重新构建一次时间轴，看看还有没有别的 API 小坑埋在路上。

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from main import BTreeBuildAndSearch\n\ntimeline = BTreeBuildAndSearch()\nbuilt = timeline.build(quiet=True)\n\nprint(type(built).name)\n\nprint(f'duration={timeline.current_time:2f}')\n\nprint(f'visible_n

Checked , no problems found

静态构建已经过了，我再用项目自己的 JAnim 命令跑一遍目标时间轴，确认 CLI 侧也能正常把这段动画建起来。

Ran terminal command: `uv run janim run main.py BTreeBuildAndSearch`

我想看一眼几个关键时间点的实际画面，确认是不是存在“只在叠加、不在回收”的视觉问题；这比凭空猜 Transform 的脾气靠谱得多。

```
已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from\npathlib import Path\nfrom main import BTreeBuildAndSearch\n\nout_dir =\nPath('debug_frames')\nout_dir.mkdir(exist_ok=True)\n\nbuilt =\nBTreeBuildAndSearch().build(quiet=True)\n\nfor idx, t in enumerate([2, 5, 9, 13, 18, 24, 30, 35],\nstart=1):\n    img = built.capture(t)\n    path = out_dir / f'frame_{idx}_{t:.0f}.png'\n    img.save(path)\n    print(path)\n"} }
```

我抓了几帧静态图出来，先抽查几个关键时间点，看分裂前后和查询阶段的画面有没有明显“越画越多”的毛病。

Viewed image

Viewed image

Viewed image

Viewed image

我抓到根因了：这里的变换语义更接近“旧物件过渡到新物件，并由新物件接管显示”，所以引用如果不切到新对象，就会一层层叠出“幽灵分身”。顺手把插入 token 的尺寸也统一掉。

Generating patch (67 lines) in

我把引用切换修好了，接着重新构建并再抓几帧对比；如果重影消失，说明这个“分身术”已经被收回去了。

```
已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from\npathlib import Path\nfrom main import BTreeBuildAndSearch\n\nout_dir =\nPath('debug_frames')\nout_dir.mkdir(exist_ok=True)\n\nbuilt =\nBTreeBuildAndSearch().build(quiet=True)\n\nprint(type(built).name)\n\nfor t in [5, 9, 18, 35]:\n    img =\n    built.capture(t)\n    path = out_dir / f'check_{t:.0f}.png'\n    img.save(path)\n    print(path)\n"} }
```

Checked, no problems found

我再抽查修复后的几帧，重点看信息面板和树节点是否还在叠影。

Viewed image

Viewed image

Viewed image

我再做一个小收口：把信息面板里的动态文案压短一点，避免末尾查询阶段的说明挤出面板边界。

Generating patch (71 lines) in

我做最后一轮构建确认，并只看收尾那一帧；如果底部说明不再顶出面板，这版就算收工。

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from pathlib import Path\nfrom main import BTreeBuildAndSearch\n\nout_dir = Path('debug_frames')\nout_dir.mkdir(exist_ok=True)\n\nbuilt = BTreeBuildAndSearch().build(quiet=True)\nning = built.capture(35)\npath = out_dir / 'final_check_35s.png'\ning.save(path)\nprint(path)\nprint(f'duration={BTreeBuildAndSearch().build(quiet=True).time}\nif hasattr(BTreeBuildAndSearch().build(quiet=True), 'timeline') else 'ok')')\n" }

Checked , no problems found

Viewed image

画面还有一个小细节：最终树的最右叶节点有点“蹭”到信息面板边框了。我把整棵树再往左挪一点，留出更舒服的呼吸空间。

Generating patch (6 lines) in

我做最后一次构建和截图确认，主要看右侧叶节点和信息面板之间的间距。

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from pathlib import Path\nfrom main import BTreeBuildAndSearch\n\nout_dir = Path('debug_frames')\nout_dir.mkdir(exist_ok=True)\n\nbuilt = BTreeBuildAndSearch().build(quiet=True)\npath = out_dir / 'spacing_check_35s.png'\nbuilt.capture(35).save(path)\nprint(path)\n" }

Checked , no problems found

Viewed image

还有一点“贴边”，我再轻推一小步，保证树和右侧面板彻底分开。最后一次了，真的——这次是像素级收尾 😊。

Generating patch (6 lines) in

我做最后的最后一次截图确认，只看右侧留白；没问题就不再折腾这棵树了。

已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from pathlib import Path\nfrom main import BTreeBuildAndSearch\n\nout_dir = Path('debug_frames')\nout_dir.mkdir(exist_ok=True)\n\nbuilt = BTreeBuildAndSearch().build(quiet=True)\npath = out_dir / 'final_spacing_35s.png'\nbuilt.capture(35).save(path)\nprint(path)\n" }

Checked , no problems found

Continue to iterate? [object Object]

已进行更改。

User: @agent Continue: "Continue to iterate?"

GitHub Copilot: Viewed image

我把树挪得有点太“谦让”了，左叶子都快出镜了。回调到一个更均衡的位置：既不挤面板，也不把左边裁掉。

Generating patch (6 lines) in

我快速再看一眼最终平衡版的收尾画面，确认左右都没被牺牲。

```
已运行 pylanceRunCodeSnippet Completed with input: { "workspaceRoot": "file:///data_deleted/btree_viz", "workingDirectory": "e:/data_deleted/btree_viz", "timeout": 30000, "codeSnippet": "from\npathlib import Path\nfrom main import BTreeBuildAndSearch\n\nout_dir =\nPath('debug_frames')\nout_dir.mkdir(exist_ok=True)\n\nbuilt =\nBTreeBuildAndSearch().build(quiet=True)\n\npath = out_dir /\n'balanced_spacing_35s.png'\nbuilt.capture(35).save(path)\nprint(path)\n"}\n
```

Checked , no problems found

Viewed image

我顺手把这次踩到的两个仓库级经验记下来，免得下次又和 JAnim 的变换语义、字体配置重新认识一遍。

Created memory file

我这边实现、修正和验证都已经收尾，现在把任务清单补到最终状态，避免留尾巴。

已完成: 设计 B 树数据与布局 (3/6)

已完成

我把 main.py 从示例圆方动画，重做成了一个完整的 B 树教学动画，核心内容包括：

- BTreeRecorder：真实执行 B 树插入，记录
 - 普通插入
 - 根分裂
 - 子节点分裂
- 统一布局函数：根据树状态自动计算节点和边的位置
- BTreeBuildAndSearch 时间轴：
 - 按步骤展示构建过程
 - 在分裂时突出中位键提升
 - 演示查询 17 命中、8 失败
- 右侧信息面板：显示当前阶段标题和简短说明

另外顺手修了两个画面问题：

- 解决了连续变换后节点/文字“重影叠加”的问题
- 让插入/查询用的移动 key token 尺寸和树里的 cell 保持一致

改了什么

main.py

现在这个文件里主要有这些部分：

- BTreeNode / SnapshotNode
 - 数据层和快照层分开
- BTreeRecorder
 - 负责生成树状态变化
- HighlightState
 - 控制节点、键、边的高亮状态
- BTreeBuildAndSearch

- 真正的 JAnim Timeline

动画内容包含：

- 插入序列：10, 20, 5, 6, 12, 30, 7, 17
- 查询演示：17 成功、8 失败

验证结果

我在当前 Windows + 项目虚拟环境里做了无界面构建验证，结果正常：

- main.py 无静态错误
- BTreeBuildAndSearch().build(quiet=True) 成功
- 构建出的时间轴总时长约 36.79s
- 我还抓了多个关键时间点的画面检查：
 - 早期插入
 - 分裂阶段
 - 查询收尾阶段

这次验证还专门确认了：

- 不再出现节点/文字层层叠影
- 信息面板说明不再糊成一团
- token 尺寸与树中 cell 一致

参考依据

这版实现是按 JAnim 官方文档的这些落点来做的，不是拍脑袋硬搓：

- get_started.html
 - Timeline -> construct()、play()、forward()
- tutorials/animations.html
 - Transform、.anim、并行动画与节奏控制
- tutorials/item_group.html
 - Group 与整体布局思路
- janim/anim/timeline.html
 - show/play/forward/prepare 的时间轴语义
- janim/items/geometry/polygon.html
 - Rect / RoundedRect
- janim/items/geometry/line.html
 - 树边连接
- janim/items/text.html
 - 说明文字与面板文本

当前清单

- [x] 梳理现有项目与参考
- [x] 抓取 JAnim 官方文档
- [x] 设计 B 树数据与布局
- [x] 实现构建与查询动画
- [x] 运行预览并修正错误
- [x] 补充说明与仓库记忆

小结

现在 main.py 已经能展示一套完整的 **B** 树构建 + 分裂 + 查询 动画流程，并且我已经把最影响观感的两个问题修掉了。

如果你愿意，我下一步可以继续帮你做两种增强版之一：

- 教学版：给每一步加更明确的“比较哪几个键 / 走哪条边”的标注
- 观赏版：把分裂时的 key 上移、兄弟节点展开做得更丝滑一点

已进行更改。